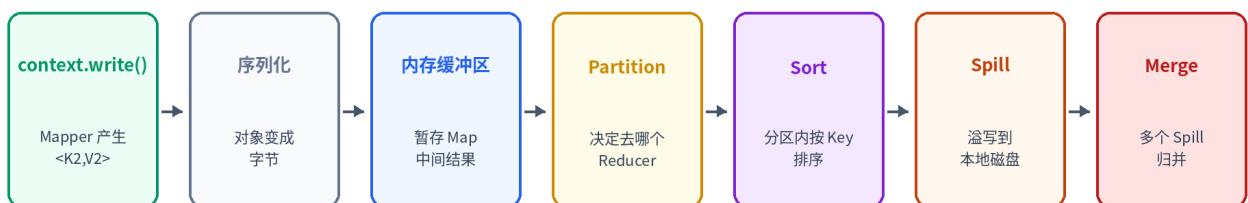


Hadoop 中的 MapReduce 原理详解

第三讲：Mapper 输出之后发生了什么

内存缓冲 · Spill · Partition · Sort · Merge

Mapper 输出之后：Map 端真正发生的流水线



最终形成：一个 MapTask 的本地中间输出文件（内部按 Reducer 分区，且每个分区按 Key 有序）

注意：这里还没有进入 Reduce。下一讲的 Shuffle，正是 Reducer 通过网络来取这些 Map 输出分区。

目标：理解 Shuffle 发生之前，Map 端已经为“按 Key 重组数据”做了哪些准备

学习目标

上一讲解决的是“一个 HDFS 文件怎样变成多个 Mapper 的输入”。这一讲从 `mapper.map()` 的另一端继续：当你的代码调用 `context.write(key, value)` 后，中间结果去了哪里？为什么不能每产生一条就通过网络发给 Reducer？为什么要先放内存、再溢写到本地磁盘、还要分区和排序？这些问题正是理解 Shuffle 的前置基础。

- 理解 Map 输出为什么是“临时中间结果”，通常不直接写 HDFS。
- 理解 `context.write()` 后发生的序列化、内存缓冲和元数据记录。
- 理解 Spill（溢写）为什么存在，以及软阈值为什么比“缓冲区装满才写盘”更合理。
- 理解 Partitioner 的职责：同一个 Key 为什么必须被送到同一个 Reducer。
- 理解 Map 端 Sort 的真实含义：先按分区组织，再在分区内部按 Key 排序。
- 理解一个 Mapper 多次 Spill 后，为什么还要 Merge 成最终本地 Map 输出。
- 知道 Combiner 在 Map 端能做什么、为什么它只是优化而不是语义保证。
- 理解 Map-only Job（Reduce 数为 0）为什么会走另一条输出路径。

本讲总链条

Mapper -> `context.write()` -> 序列化 -> 内存排序缓冲区 -> Partition -> Sort -> Spill -> 多路 Merge -> 本地 Map 输出。下一讲 Reducer 再通过网络拉取自己对应的分区。

0. 和上一讲接起来：map() 只是业务函数，不负责保存结果

上一讲我们看到，Hadoop 框架不断从 `RecordReader` 取出 `<K1,V1>`，然后调用你的 `map()`。你的 Mapper 通常会在处理一条记录时调用 `context.write(K2,V2)` 产生零条、一条或多条中间记录。这里有一个关键点：`context.write()` 并不等于“立刻写入最终文件”。



如果你把 `context.write()` 理解成普通 Java 的 `println()` 或“直接写文件”，后面很多机制都会理解错。它更像“把一条中间记录交给 MapReduce 框架”，至于怎样缓存、排序、落盘、传给哪个 Reducer，由框架接管。

1. 为什么 Map 输出不直接写 HDFS?

1.1 Map 输出通常不是最终结果

以 WordCount 为例，一个 Mapper 可能产生大量 `<word,1>`。这些记录还没有把相同单词聚合到一起，因此只是中间状态。最终结果要等所有 Mapper 的同 Key 数据汇合到 Reducer 后才能确定。把这种短生命周期的中间结果先写 HDFS，会带来不必要的网络、NameNode 元数据和多副本写入开销。

1.2 Map 输出通常先写“执行节点本地磁盘”

当 Job 有 Reducer 时，Map 输出在完成内存缓冲、Spill 和归并后，通常保存在运行该 MapTask 的节点本地磁盘中。它是可重算的临时数据：如果丢了，可以重新执行对应 MapTask，而不是像原始输入或最终结果那样依赖 HDFS 持久保存。

数据类型	典型存放位置	为什么
原始输入	HDFS	需要可靠持久保存，多副本容错
Map 中间输出	Map 节点本地磁盘	临时、可重算，避免 HDFS 的持久化成本
Reduce 最终输出	HDFS	是 Job 需要保留和供后续使用的结果

底层原则

昂贵的可靠持久化只给“必须保留的数据”；可重算的中间数据更适合用本地临时存储。这是很多分布式系统都会采用的成本权衡。

2. context.write() 之后第一步：序列化

2.1 Mapper 输出的是 Java 对象，磁盘和网络处理的是字节

Mapper 里的 Key/Value 可能是 Text、IntWritable、LongWritable 或自定义 Writable 对象。框架若要把它们放入缓冲区、写入磁盘、以后再通过网络传给 Reducer，就必须先把对象转换成紧凑字节表示，这个过程叫 Serialization（序列化）。

```
context.write(new Text("cat"), new IntWritable(1));
```

```

逻辑对象:  <Text("cat"), IntWritable(1)>
           | Writable 序列化
           v
字节数据:  [key bytes][value bytes] + 元数据索引

```

为什么 Hadoop 自己提供 Writable 体系？一个重要原因就是高频分布式数据交换需要尽量低开销、可控的二进制序列化。MapReduce 中间结果数量可能达到数百万、数亿条，单条多几个字节或多一次昂贵对象转换，累计成本都会很明显。

3. 内存排序缓冲区：为什么不每条记录都写磁盘？

3.1 批量处理比逐条同步写盘高效

如果 Mapper 每产生一条 <K2,V2> 就立刻进行一次磁盘写入，会产生大量随机或细碎 I/O，吞吐量很差。Hadoop 的做法是先在内存中批量收集 Map 输出，同时记录每条记录的分区、Key/Value 位置等元数据，达到一定条件后再成批 Spill 到磁盘。

为什么需要 Spill：Map 输出先进入内存，但内存不可能无限大

示意：排序缓冲区不断接收序列化后的 <Key,Value> 和元数据索引。达到软阈值后，后台线程开始溢写。



软阈值的意义：尽量让 Mapper 一边继续收集新输出，一边由后台线程把旧数据写盘，而不是每条记录都同步写磁盘。

3.2 “100 MB、80%” 应该怎么理解？

在 Hadoop 3.x 的经典默认配置中，`mapreduce.task.io.sort.mb` 常见默认值是 100 MB，用于控制 Map 输出序列化与记录元数据缓冲区的累计规模；`mapreduce.map.sort.spill.percent` 是 Spill 的软触发比例，经典默认值为 0.80。集群管理员可以覆盖这些配置，所以不要把具体数值当成算法定义。真正要掌握的是“有限内存 + 软阈值 + 后台溢写”这套机制。

示意（假设 `sort buffer = 100 MB`，`spill.percent = 0.80`）：

已使用约 80 MB -> 触发后台 Spill 线程
Mapper/collector 尽量继续收集后续输出
Spill 线程同时对待写数据做 partition/sort，并写入本地磁盘

容易误解

达到 80% 并不等于 Mapper 必须马上停住等磁盘写完。这个阈值更像“启动后台清理工作的触发器”。当 Spill 来不及、缓冲空间真的吃紧时，生产者才可能被迫等待。

4. Partition：每条 Map 输出到底属于哪个 Reducer？

4.1 Reduce 并行的前提：先把 Key 空间切成若干分区

假设 Job 配置了 4 个 Reducer。每条 Map 输出都必须被分配到 0、1、2、3 中的某一个 partition。之后 Reducer 0 只取 partition 0，Reducer 1 只取 partition 1，以此类推。因此，Partitioner 实际上定义了“Key 如何映射到 Reducer”。

4.2 默认 HashPartitioner

默认 HashPartitioner 的核心思想是对 Key 求 hash，再对 Reducer 数取模。典型公式可以写成：

```
partition = (key.hashCode() & Integer.MAX_VALUE) % numReduceTasks
```

& Integer.MAX_VALUE 的目的之一是把符号位处理掉，得到非负分区索引。最关键的性质不是“hash”本身，而是：相同 Key 在相同 Reducer 数配置下会得到相同 partitionId，所以同一个 Key 才有机会在 Reduce 端汇总到一起。

MapReduce 的硬约束

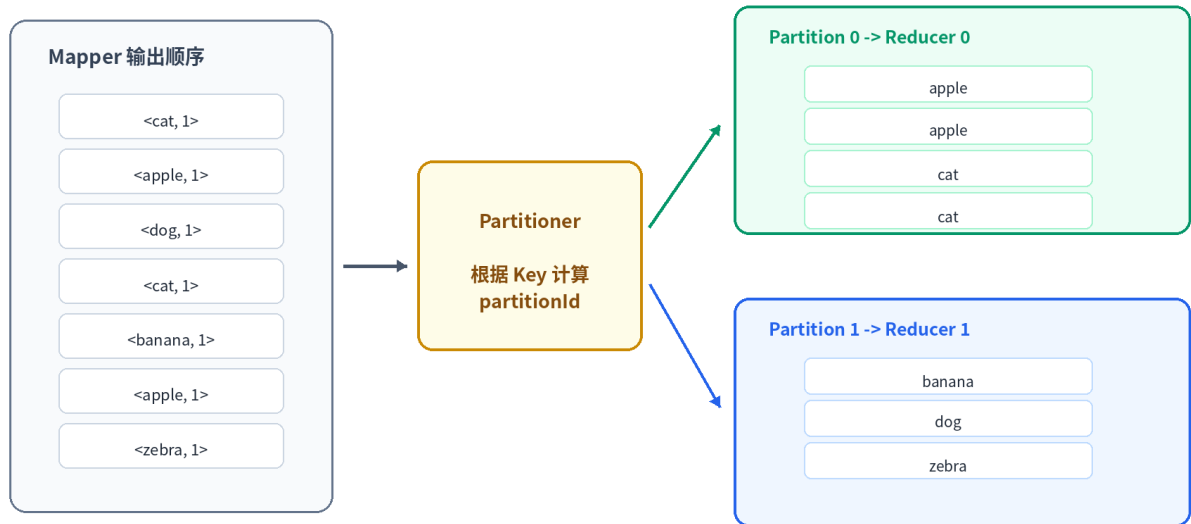
如果同一个 Key 被送到两个不同 Reducer，那么两个 Reducer 都只看见它的一部分数据，最终聚合就会错误。因此“相同 Key 必须进同一分区”是正确性的要求，不只是性能优化。

4.3 Partitioner 也可能造成数据倾斜

Hash 通常能让大量不同 Key 大致均匀分散，但它不能保证“记录数量”或“处理成本”均匀。如果某个热点 Key 本身占了 40% 的记录，那么这个 Key 无论怎么 hash 都必须整体进入同一个 Reducer，于是该 Reducer 会成为长尾瓶颈。这就是后面要专门学习的数据倾斜。

5. Sort: Map 端为什么必须排序?

Partition + Sort：先决定“去哪个 Reducer”，再在每个分区内按 Key 排序



关键：排序不是把“所有 Reducer 的数据混成一个全局有序表”，而是让每个分区内部有序。

5.1 准确说法：先按分区组织，再在分区内部按 Key 排序

初学者常听到“Map 输出会排序”，容易以为整个集群所有 Mapper 输出先做一次全局排序。不是。每个 Mapper 只处理自己的中间结果；在 Map 端，它会让不同 Reducer 对应的 partition 形成各自连续区域，并让每个 partition 内部按 Key 有序。

5.2 为什么排序对 Reduce 端这么有价值?

因为 Reducer 未来要把来自很多 Mapper 的同 Key 数据合并到一起。如果每个 Mapper 提供的分区已经按 Key 排好序，Reducer 就可以采用多路归并（merge sort）的方式边拉取边合并，不需要把所有记录先完全打乱再重新建立巨大哈希表。

Mapper A 的 partition 0: apple, apple, cat, dog
 Mapper B 的 partition 0: apple, banana, cat, cat
 Mapper C 的 partition 0: apple, cat, dog, zebra

Reducer 可以对这些“已有序流”做多路归并：

```
apple -> banana -> cat -> dog -> zebra
```

本质

Map 端排序是在提前付出一些 CPU 与磁盘排序成本，换取 Reduce 端更容易做大规模流式归并和按 Key 分组。

5.3 同一个 Key 的 Value 顺序有保证吗？

默认语义里，MapReduce 主要保证 Key 的排序与分组；同一个 Key 对应的多个 Value 之间通常不要依赖“原始输入顺序”。如果业务确实需要 Value 内部的特定顺序，需要使用复合 Key、排序比较器和分组比较器等“二次排序（Secondary Sort）”技巧，这属于后续高级内容。

6. Spill：把一批已组织好的中间结果写到本地磁盘

6.1 Spill 时不只是“把内存复制到硬盘”

Spill 线程拿到一批待写数据后，会根据 partition 组织记录，并按 Key 排序，然后把结果写成本地 spill 文件。若配置了 Combiner，框架还可能在合适阶段对同 Key 做一次本地聚合，从而减少后续需要保留和传输的记录量。

内存中的 Map 输出

```
|
+--> Partition 0 --按 Key 排序--+
+--> Partition 1 --按 Key 排序--+--> spill_N.out (本地磁盘)
+--> Partition 2 --按 Key 排序--+
```

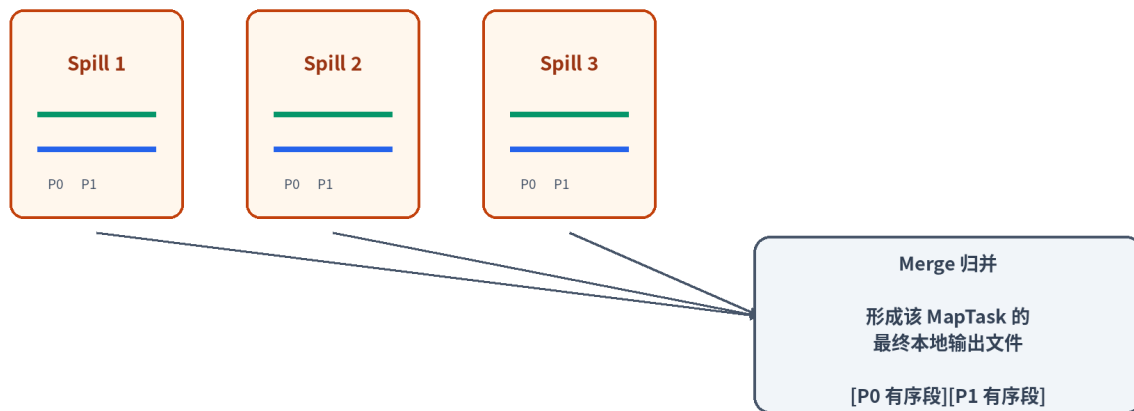
一个 spill 文件内部通常包含多个 partition 的连续有序段。框架还需要索引信息来记录每个 partition 段在文件中的起始位置和长度，这样后面某个 Reducer 才能只取属于自己的那一段。

6.2 为什么一个 Mapper 会产生多个 Spill 文件？

因为 MapTask 可能持续产生远超内存缓冲区容量的中间结果。缓冲区达到阈值就 Spill 一批，然后继续接收新输出；过一会儿再次达到阈值，再 Spill 第二批。一个处理 128 MB 输入的 Mapper，并不意味着它只输出 128 MB；某些业务会“膨胀”，一条输入可能产生多条中间记录，因此 Spill 次数可能很多。

7. Merge：为什么 Spill 完了还没结束？

一个 Mapper 可能 Spill 多次：Map 结束时还要做本地归并



Reducer 以后只取属于自己的那一段。例如 Reducer 1 只拉取每个 Mapper 的 P1，而不是把整个 Map 输出文件全部复制过去。

7.1 多个 Spill 文件不能直接留给 Reducer 自己收拾

如果一个 Mapper 最后留下几十个零散 spill 文件，那么每个 Reducer 都要从这个 Mapper 管理许多小段，连接、索引和磁盘读取成本会很高。因此 MapTask 结束前会把这些 spill 段进行归并，形成该 MapTask 的最终本地 Map 输出。

7.2 归并后仍然按 partition 分段

归并后的文件不是“所有 partition 混成一锅”。它仍然保留按 Reducer 分区的边界，并且每个分区中的 Key 有序。这样下一阶段 Reducer 1 可以准确读取每个 Mapper 的 partition 1，Reducer 2 读取 partition 2。

非常重要

一个 Mapper 的最终 Map 输出文件可以同时服务多个 Reducer；但每个 Reducer 只会拉取其中属于自己的 partition 段。

8. Combiner：Map 端“可选的局部 Reduce”

8.1 它为什么能显著减少 Shuffle 数据量？

WordCount 中，一个 Mapper 可能产生 10 万条 <the,1>。如果这些记录原封不动进入 Shuffle，就要通过磁盘和网络处理 10 万条。若配置合法 Combiner，可以在 Map 节点先把它们合成 <the,100000>，网络中只需传很少的记录。

没有 Combiner:

<the,1> <the,1> <the,1> ... 共 100000 条

| Shuffle

v

Reducer 再求和

有 Combiner:

<the,1> x 100000 -> Map 本地先合成 <the,100000>

| Shuffle

v

Reducer 再与其他 Mapper 的局部和相加

8.2 为什么 Combiner 不能参与业务正确性假设?

因为框架不保证 Combiner 对每条数据、每次 Spill 都以固定次数执行。它可能执行 0 次、1 次或多次。因此你的算法必须满足：即使完全不执行 Combiner，最终 Reducer 结果仍然正确。

Combiner 只是优化器，不是逻辑必经阶段。

8.3 为什么“求和”适合，“直接求平均值”容易出错?

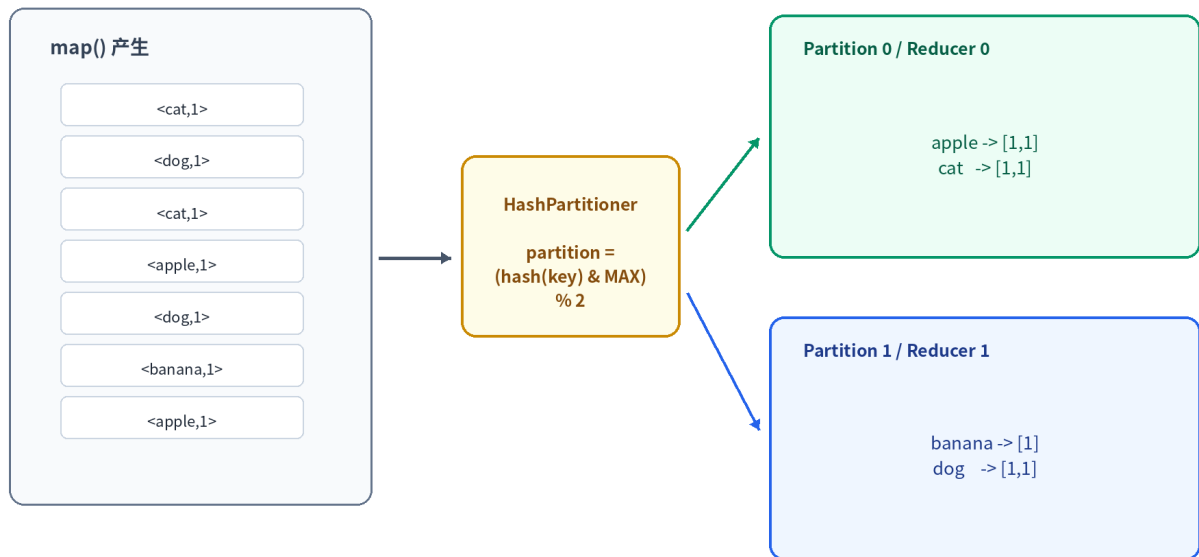
求和具有良好的可结合性：先把局部 1+1+1 合成 3，再和别处的 4 相加得到 7，与一次性全部相加等价。但平均值 $\text{avg}(\text{avg}(A), \text{avg}(B))$ 在两组样本数量不同的时候不等于总体平均值。正确做法通常是传递 <sum,count>，最后再算 sum/count。

判断 Combiner 的通用思路

问自己：把输入任意切成几组，各组先局部聚合，再把局部结果继续用同一个规则聚合，最终是否仍和一次性全量聚合等价？如果不等价，就不能简单把 Reducer 当 Combiner。

9. WordCount：把本讲所有机制串起来

WordCount 例子：同一个 Mapper 的中间结果如何变成两个 Reducer 的分区



这里的具体分区只用于演示；真实 HashPartitioner 的结果取决于 Key 的 hashCode。真正不变的是：相同 Key 必须进入同一个分区。

假设某 Mapper 处理自己的 InputSplit，依次输出 cat、dog、cat、apple、dog、banana、apple。现在 Job 配置 2 个 Reducer。注意图中的具体 Key 到 partition 的映射只是为了演示，真实结果由 Key 的 hashCode 决定。

1. Mapper 调用 context.write(word, 1)，框架把 Key/Value 序列化后放进内存排序缓冲区。
2. Partitioner 为每条记录计算 partitionId，决定以后属于 Reducer 0 还是 Reducer 1。
3. 缓冲区达到 Spill 阈值后，后台线程开始处理当前批次。
4. 每个 partition 内部按 Key 排序，因此相同 Key 在有序序列中会相邻。
5. 如果有 Combiner，可能把相邻的 <cat,1><cat,1> 局部合成 <cat,2>。
6. 这一批数据写到 spill 文件；后续输出继续进入缓冲区，可能产生第二、第三个 spill。
7. MapTask 完成后，把多个 spill 归并成最终本地 Map 输出，保留 partition 边界和 Key 有序性。
8. 下一阶段 Reducer 0 从所有 Mapper 拉取 partition 0，Reducer 1 从所有 Mapper 拉取 partition 1。

10. 特殊情况：如果 Reducer 数量是 0 呢？

10.1 Map-only Job 不需要 Shuffle

如果 Job 设置 numReduceTasks = 0，那么 Mapper 的输出本身就是最终输出。此时没有“把 Key 分给不同 Reducer”的需求，也没有后续 Reduce 端 Shuffle，因此 Map 输出会通过 OutputFormat 直接写最终输出位置，而不是为了 Shuffle 去构造同样的 partition/sort 中间文件。

```
job.setNumReduceTasks(0);
```

输入 -> Mapper -> OutputFormat -> HDFS 最终结果
X 不需要 Reducer / Shuffle

这个边界条件非常重要，因为它提醒我们：Partition、Map 端 Sort、Spill/Merge 这一整套经典路径，是为“存在 Reduce 阶段、需要跨 Mapper 按 Key 重组数据”服务的。MapReduce 框架不是为了形式完整而强迫每个 Job 都经过 Shuffle。

11. 性能视角：Map 端哪些指标最值得关注？

现象	底层原因	通常意味着什么
Spill 次数很多	缓冲区相对输出量太小，或 Map 输出膨胀	更多本地磁盘 I/O 与归并开销
Map output bytes 很大	Mapper 产生大量中间数据	后续 Shuffle 网络成本可能很高
某个 partition 特别大	热点 Key 或分区不均匀	未来某个 Reducer 可能成为长尾
Combiner 后记录数大幅下降	局部可聚合程度高	可以显著减少磁盘与网络数据量
Map 输出压缩后明显变小	数据可压缩性高	可能用 CPU 换网络/磁盘收益
单条记录极大	Key/Value 本身异常大	缓冲、序列化和 Spill 都可能受影响

真正优化 MapReduce 时，不能只盯着“Mapper 跑了多久”。Map 输出量、Spill 次数、Shuffle bytes、Reducer 分区大小这些指标往往更能解释系统为什么慢。

12. 几个关键配置参数：先认识，不要求死记

配置项	作用	理解重点
mapreduce.task.io.sort.mb	控制 Map 输出排序/序列化缓冲区规模	大一些可能减少 Spill，但会占用更多 Mapper 内存
mapreduce.map.sort.spill.percent	控制 Spill 软触发阈值比例	阈值是触发点，不是业务逻辑
mapreduce.job.reduces / setNumReduceTasks	Reducer 数量	直接影响 partition 数和 Reduce 并行度
mapreduce.job.partitioner.class	自定义 Partitioner	控制 Key -> Reducer 的映射
mapreduce.map.output.compress	是否压缩 Map 中间输出	CPU 换磁盘与网络带宽，具体收益取决于数据与 codec

不要先调参

初学阶段不要把学习重点放在“100 MB 改成多少最好”。先掌握数据流和因果关系：输出多 -> 缓冲满 -> Spill 多 -> 磁盘 I/O 增；分区偏 -> Reducer 倾斜；中间数据大 -> Shuffle 网络成本高。知道机制以后，参数才有意义。

13. 初学者最常见的 14 个疑问

Q1: context.write() 会直接把数据写到 HDFS 吗？

有 Reducer 的典型 Job 中不会。它先进入 Map 端中间输出处理流程；最终 Reduce 输出才写 HDFS。

Q2: 为什么不直接把 Mapper 输出发送给 Reducer？

因为 Map 会持续产生大量细粒度记录，直接逐条网络发送会导致大量小消息；而且 Reducer 需要按 Key 组织数据，Map 端先分区、排序和批量落盘更高效。

Q3: Spill 是“内存溢出异常”吗？

不是。这里的 Spill 是框架设计好的正常外排机制，用有限内存处理任意大的中间输出。

Q4: 达到 Spill 阈值后 Mapper 就完全停止吗？

不一定。Spill 通常由后台线程执行，收集线程在空间允许时可继续工作；真正空间不足时才会等待。

Q5: Partition 和 HDFS Block 有关系吗？

几乎是两个不同维度。Block/Split 解决输入如何切；Partition 解决 Map 输出以后交给哪个 Reducer。

Q6: Reducer 数量为 4，是不是每个 Mapper 都会产生 4 个独立文件？

逻辑上有 4 个 partition，但最终 Map 输出通常可把多个分区段组织在一个本地输出文件中并用索引定位，不必理解成 4 个永久独立文件。

Q7: 不同 Mapper 对同一个 Key 会算出相同 partition 吗？

在使用相同 Partitioner 与相同 Reducer 数时应该如此，这正是同 Key 能汇合到同一 Reducer 的基础。

Q8: Map 端 Sort 是全局排序吗？

不是，是每个 Mapper 自己的输出按 partition 组织，并在 partition 内按 Key 排序。

Q9: 为什么先 Partition 再 Sort？

因为最终目标是让每个 Reducer 获得自己的有序数据段。排序只需要在目标分区内部有意义。实现内部会把 partitionId 和 key 排序信息结合处理。

Q10: Spill 文件写到 HDFS 吗？

通常写到 MapTask 所在节点的本地临时磁盘。

Q11: MapTask 成功后本地输出丢了怎么办？

Reduce 获取不到时，框架可以让对应 MapTask 重新执行并重新生成中间输出；这就是“可重算临时数据”的思路。

Q12: Combiner 一定会执行吗？

不保证。业务正确性绝不能依赖 Combiner。

Q13: 设置 1 个 Reducer 时还需要 Partition 吗？

逻辑上所有 Key 都属于唯一分区，分区选择变得很简单，但排序、Spill/merge 仍然服务于 Shuffle/Reduce。

Q14: 没有 Reducer 时还会 Sort/Shuffle 吗？

典型 Map-only Job 不需要按 Reducer 分区和 Shuffle，Mapper 输出直接作为最终输出。

14. 迁移练习：把机制迁移到日志统计

任务：统计 100 GB Web 日志中每个 IP 的访问次数。Mapper 对每条日志解析 IP，并输出 <IP,1>。假配置 8 个 Reducer。

问题 1：为什么 Mapper 不直接维护一个“全局 IP -> 次数”的 HashMap？

参考答案

因为每个 Mapper 只看见自己的 Split，而且全局 IP 种类可能极多；MapReduce 的核心就是先做局部产生，再把同 Key 跨 Mapper 重组。

问题 2：为什么同一个 IP 必须经过 Partitioner 进入同一个 Reducer？

参考答案

因为最终次数要汇总所有 Mapper 中这个 IP 的贡献。分到两个 Reducer 会得到两个不完整计数。

问题 3：如果 10.0.0.1 占全部流量 50%，HashPartitioner 能解决倾斜吗？

参考答案

不能。相同 Key 必须整体进同一 Reducer，因此这个热点 Key 会让对应 Reducer 极重，需要业务层面的分片/两阶段聚合等办法。

问题 4：Combiner 为什么在这个任务里很适合？

参考答案

同一个 Mapper 内可以先把 <IP,1> 聚成 <IP,localCount>，最终 Reducer 对局部计数继续求和，语义不变。

问题 5：如果 Map 输出 300 GB，但输入只有 100 GB，说明什么？

参考答案

说明 Mapper 产生了输出膨胀；后续 Spill、磁盘 I/O 和 Shuffle 网络都会很重，应检查业务输出粒度或能否本地聚合。

15. 本讲最终心智模型

层次	核心问题	机制
产生	Mapper 输出了什么	context.write(K2,V2)
表示	对象怎样变成可落盘/传输的数据	序列化
暂存	为什么先放内存	批量化、减少细碎 I/O
分流	以后去哪个 Reducer	Partitioner
组织	怎样让 Reduce 更容易归并分组	partition 内按 Key 排序
外排	内存不够怎么办	Spill 到本地磁盘
收敛	多个 spill 怎么办	本地 Merge 成最终 Map 输出
优化	能否先减少中间数据	可选 Combiner / 压缩

如果只记一句话，可以记成：Mapper 不是把结果“直接交给 Reducer”，而是先把中间结果在本地组织成“按 Reducer 分区、分区内按 Key 有序”的可传输数据结构。Shuffle 就是在这个基础上，把每个 Reducer 需要的分区从所有 Mapper 拉过来。

核心因果链

Key 决定 partition -> partition 决定目标 Reducer -> 排序让同 Key 靠拢 -> Spill 解决内存有限 -> Merge 减少碎片 -> Shuffle 再把对应分区跨网络汇集。

附录：本讲术语速查

术语	通俗解释
Map Output	Mapper 产生、等待 Reduce 继续处理的中间 Key-Value。
Serialization	把 Key/Value 对象转换成紧凑字节表示。
Sort Buffer	Map 端暂存并组织中间输出的内存缓冲结构。
Spill	达到阈值后，把一批内存中间结果排序并写入本地磁盘。
Partition	一个 Reducer 对应的数据分区。
Partitioner	根据 Key 决定 partitionId 的组件。
HashPartitioner	默认常用分区器，通过 Key hash 与 Reducer 数映射分区。
Map-side Sort	每个 Mapper 内部把各 partition 的 Key 排成有序数据。
Merge	把多个 spill 有序段归并为更少、最终的本地 Map 输出。
Combiner	可选的 Map 本地聚合优化，不保证执行。
Map-only Job	Reducer 数为 0，Map 输出直接成为最终结果的 Job。

下一讲预告：真正进入 Shuffle

下一讲我们会从 Reducer 的视角继续：Reducer 怎样知道哪些 Mapper 已经完成？怎样通过网络从所有 Mapper 节点拉取自己的 partition？为什么 Shuffle 与 Sort 是交叠发生的？Reduce 端内存放不下怎么办？多个 Map 输出怎样 Merge、Group，最后怎样把 key -> [v1,v2,...] 交给 reduce()。这是 MapReduce 最核心、也是性能代价最大的阶段。

参考依据

本讲机制以 Apache Hadoop 3.x MapReduce 经典实现为背景，配置名与执行阶段参考 Apache Hadoop 官方 MapReduce Tutorial、Reducer API 以及 Hadoop 3.3.5 集群配置文档。具体默认参数可由集群管理员覆盖，因此学习重点应放在机制和因果关系，而不是死记数值。